Tarea M43

Alumno: Yamil Salomón

Práctica: Tópicos avanzados

Punto 1: Propuesta de Base de Datos y Diccionario de Datos *Contexto de Negocio:* Se requiere analizar el comportamiento de compra de los usuarios de una plataforma de e-commerce. Para ello, se estructuraron tres tablas principales que permiten relacionar quién compra, qué compra y en qué estado se encuentra la transacción.

Diccionario de Datos:

1. **clientes:**
    - id_cliente (INT): Identificador único del cliente.
    - nombre (TEXT): Nombre completo del cliente.
    - pais (TEXT): País de residencia.
2. **productos:**
    - id_producto (INT): Identificador único del producto.
    - categoria (TEXT): Categoría a la que pertenece el artículo.
    - precio (FLOAT): Precio unitario del producto.
3. **ordenes:**
    - id_orden (INT): Identificador único de la transacción.
    - id_cliente (INT): Llave foránea que conecta con la tabla clientes.
    - id_producto (INT): Llave foránea que conecta con la tabla productos.
    - cantidad (INT): Unidades compradas en la orden.

Explicación de Negocio:

En el siguiente bloque de Python, primero estamos "fabricando" nuestros archivos CSV simulando la información que un Ingeniero de Datos nos entregaría. Inmediatamente después, creamos una conexión a una base de datos local llamada ecommerce.db usando SQLite. Finalmente, leemos esos CSVs con Pandas y los inyectamos como tablas dentro de nuestra base de datos para que estén listas para ser consultadas con SQL.

```python
import pandas as pd
import sqlite3

data_clientes = {
    'id_cliente': [1, 2, 3, 4, 5],
    'nombre': ['Ana Lopez', 'Carlos Ruiz', 'Diana Torres', 'Eduardo Paz', 'Fernanda Lima'],
    'pais': ['Mexico', 'Colombia', 'Mexico', 'Argentina', 'Chile']
}

data_productos = {
    'id_producto': [101, 102, 103, 104],
    'categoria': ['Electronica', 'Ropa', 'Hogar', 'Deportes'],
    'precio': [1200.50, 450.00, 800.00, 600.00]
}

data_ordenes = {
    'id_orden': [1001, 1002, 1003, 1004, 1005],
    'id_cliente': [1, 2, 1, 4, 99],
    'id_producto': [101, 103, 102, 101, 104],
    'cantidad': [1, 2, 3, 1, 5]
}
df_clientes = pd.DataFrame(data_clientes)
df_productos = pd.DataFrame(data_productos)
df_ordenes = pd.DataFrame(data_ordenes)

df_clientes.to_csv('clientes.csv', index=False)
df_productos.to_csv('productos.csv', index=False)
df_ordenes.to_csv('ordenes.csv', index=False)

conn = sqlite3.connect('ecommerce.db')

df_clientes_csv = pd.read_csv('clientes.csv')
df_productos_csv = pd.read_csv('productos.csv')
df_ordenes_csv = pd.read_csv('ordenes.csv')

df_clientes_csv.to_sql('clientes', conn, if_exists='replace', index=False)
df_productos_csv.to_sql('productos', conn, if_exists='replace', index=False)
df_ordenes_csv.to_sql('ordenes', conn, if_exists='replace', index=False)

print("Base de datos 'ecommerce.db' y tablas creadas exitosamente a partir de los CSV.")
```

```
Base de datos 'ecommerce.db' y tablas creadas exitosamente a partir de los CSV.
```

Name	Type	Schema
Tables (3)		
clientes		CREATE TABLE "clientes" ("id_cliente" INTEGER, "nombre" TE
ordenes		CREATE TABLE "ordenes" ("id_orden" INTEGER, "id_cliente"
productos		CREATE TABLE "productos" ("id_producto" INTEGER, "catego
Indices (0)		
Views (0)		
Triggers (0)		

Punto 2: Consulta aplicando INNER JOIN

Consulta SQL: Se requiere un reporte limpio de ventas que muestre el nombre del cliente, el ticket de la orden, la categoría del producto que llevó, la cantidad y el total pagado (cantidad * precio).

Explicación de Negocio: A diferencia del INNER JOIN que es exclusivo, el LEFT JOIN es inclusivo con la tabla "izquierda" (la principal). En este caso, al poner a clientes a la izquierda, le estamos diciendo a SQL: "Tráeme a todos los clientes sí o sí, y si encuentras que tienen órdenes en la tabla derecha, pégalas. Si no tienen órdenes, rellena esos espacios con valores nulos". Gracias a esto, detectaremos

rápidamente a clientes como 'Diana Torres' y 'Fernanda Lima', que son leads fríos y necesitan un empujón para hacer su primera compra.

```
query_inner = """
SELECT
    c.nombre AS Nombre_Cliente,
    o.id_orden,
    p.categoria AS Producto_Comprado,
    o.cantidad,
    (o.cantidad * p.precio) AS Total_Pagado
FROM clientes c
INNER JOIN ordenes o ON c.id_cliente = o.id_cliente
INNER JOIN productos p ON o.id_producto = p.id_producto;
"""

df_inner = pd.read_sql_query(query_inner, conn)
display(df_inner)
```

	Nombre_Cliente	id_orden	Producto_Comprado	cantidad	Total_Pagado
0	Ana Lopez	1001	Electronica	1	1200.5
1	Ana Lopez	1003	Ropa	3	1350.0
2	Carlos Ruiz	1002	Hogar	2	1600.0
3	Eduardo Paz	1004	Electronica	1	1200.5

Punto 3: Ejemplo de consulta con LEFT JOIN *Consulta SQL:* Se requiere un reporte que liste a toda la base de clientes actual y muestre sus órdenes de compra asociadas. Si el cliente no ha realizado ninguna compra, el reporte debe evidenciarlo para que el equipo de Marketing pueda enviarles una campaña de retargeting (correos de recuperación).

Explicación de Negocio: A diferencia del INNER JOIN que es exclusivo, el LEFT JOIN es inclusivo con la tabla "izquierda" (la principal). En este caso, al poner a clientes a la izquierda, le estamos diciendo a SQL: "Tráeme a todos los clientes sí o sí, y si encuentras que tienen órdenes en la tabla derecha, pégalas. Si no tienen órdenes, rellena esos espacios con valores nulos". Gracias a esto, detectaremos rápidamente a clientes como 'Diana Torres' y 'Fernanda Lima', que son leads fríos y necesitan un empujón para hacer su primera compra.

```
query_left = """
SELECT
    c.id_cliente,
    c.nombre AS Nombre_Cliente,
    o.id_orden,
    o.cantidad
FROM clientes c
LEFT JOIN ordenes o ON c.id_cliente = o.id_cliente;
"""

df_left = pd.read_sql_query(query_left, conn)
display(df_left)
```

	id_cliente	Nombre_Cliente	id_orden	cantidad
0	1	Ana Lopez	1001.0	1.0
1	1	Ana Lopez	1003.0	3.0
2	2	Carlos Ruiz	1002.0	2.0
3	3	Diana Torres	NaN	NaN
4	4	Eduardo Paz	1004.0	1.0
5	5	Fernanda Lima	NaN	NaN

Punto 4: Ejemplo de consulta con CASE WHEN *Consulta SQL:* El equipo de ventas necesita clasificar el catálogo de productos actual en tres niveles de precios ('Económico', 'Estándar' y 'Premium') para armar paquetes promocionales.

Explicación de Negocio *(Problema que resuelve):* La instrucción CASE WHEN actúa como un condicional lógico (un IF/ELSE dentro de SQL). Nos permite crear nuevas columnas "al vuelo" basadas en reglas de negocio sin modificar la tabla original. El problema que resuelve aquí es la **agrupación categórica**: en lugar de que un analista tenga que revisar precio por precio, el sistema etiqueta automáticamente el producto. Esto agiliza la creación de reportes y facilita la segmentación del catálogo.

```
query_case = """
SELECT
    id_producto,
    categoria,
    precio,
    CASE
        WHEN precio < 500 THEN 'Económico'
        WHEN precio BETWEEN 500 AND 1000 THEN 'Estándar'
        ELSE 'Premium'
    END AS Rango_Precio
FROM productos;
"""

df_case = pd.read_sql_query(query_case, conn)
display(df_case)
```

	id_producto	categoria	precio	Rango_Precio
0	101	Electronica	1200.5	Premium
1	102	Ropa	450.0	Económico
2	103	Hogar	800.0	Estándar
3	104	Deportes	600.0	Estándar

Punto 5: Ejemplo de consulta con SubQuery 1 (Semi-Join) *Consulta SQL:* El equipo de auditoría necesita un listado con los datos personales **únicamente** de los clientes que han realizado al menos una compra registrada en el sistema. No quieren ver los datos de las órdenes, solo la información de los clientes comprobados.

Explicación de Negocio *(Lógica detrás del Semi-Join):* La lógica del Semi-Join usando IN actúa como un "filtro de lista VIP". La subconsulta (lo que está entre paréntesis) primero genera una lista con todos los id_cliente que existen en la tabla de órdenes (nuestra "lista VIP"). Luego, la consulta principal revisa la tabla de clientes y dice: "Solo te voy a mostrar a este cliente si su ID está dentro de esa lista". A diferencia de un INNER JOIN, el Semi-Join solo devuelve columnas de la tabla principal y no duplica los datos del cliente si este ha hecho múltiples compras (como Ana López).

```
query_semi_join = """
SELECT
    id_cliente,
    nombre,
    pais
FROM clientes
WHERE id_cliente IN (
    SELECT id_cliente
    FROM ordenes
);
"""

df_semi_join = pd.read_sql_query(query_semi_join, conn)
display(df_semi_join)
```

	id_cliente	nombre	pais
0	1	Ana Lopez	Mexico
1	2	Carlos Ruiz	Colombia
2	4	Eduardo Paz	Argentina

Punto 6: Ejemplo de consulta con SubQuery 2 (Anti-Join) *Consulta SQL:* El equipo de Marketing y Retención ha solicitado una lista de los usuarios registrados que jamás han realizado una transacción. El objetivo es enviarles un código de descuento agresivo para incentivar su primera compra.

Explicación de Negocio *(Lógica detrás del Anti-Join):* El Anti-Join se construye utilizando la instrucción NOT IN. Su lógica es literalmente la inversa de la consulta anterior: primero genera la lista de clientes que existen en la tabla de órdenes, y luego le dice a la base de datos: "Filtra la tabla de clientes y devuélveme SOLAMENTE a aquellos cuyos IDs **NO** figuren en esta lista de compradores". Es una forma mucho más eficiente de encontrar "ausencias" de datos que usar un LEFT JOIN y luego filtrar por nulos.

```
query_anti_join = """
SELECT
    id_cliente,
    nombre,
    pais
FROM clientes
WHERE id_cliente NOT IN (
    SELECT id_cliente
    FROM ordenes
);
"""

df_anti_join = pd.read_sql_query(query_anti_join, conn)
display(df_anti_join)
```

	id_cliente	nombre	pais
0	3	Diana Torres	Mexico
1	5	Fernanda Lima	Chile

Punto 7: Ejemplo de consulta con Subconsulta Escalar *Consulta SQL:* El equipo de compras y catálogo quiere saber cuáles son los productos de nuestro inventario que se consideran "premium" en función del mercado actual. Para ello, solicitan un listado de los productos cuyo precio esté **por encima del precio promedio** de todo el catálogo.

Explicación de Negocio *(Lógica detrás de la Subconsulta Escalar):* En lugar de que un analista calcule el promedio a mano (ej. calculando que el promedio es 762.62) y luego haga un WHERE precio > 762.62, anidamos una consulta que hace ese cálculo en tiempo real: (SELECT AVG(precio) FROM productos). La ventaja de negocio es que esta consulta es 100% automatizada y escalable; si mañana agregamos 50 productos nuevos al inventario, el promedio se recalculará solo y la consulta siempre nos dará la lista correcta de productos por encima del nuevo ticket promedio.

```
query_sub_escalar = """
SELECT
    id_producto,
    categoria,
    precio
FROM productos
WHERE precio > (
    SELECT AVG(precio)
    FROM productos
);
"""

df_sub_escalar = pd.read_sql_query(query_sub_escalar, conn)
display(df_sub_escalar)
```

	id_producto	categoria	precio
0	101	Electronica	1200.5
1	103	Hogar	800.0